

Pointers

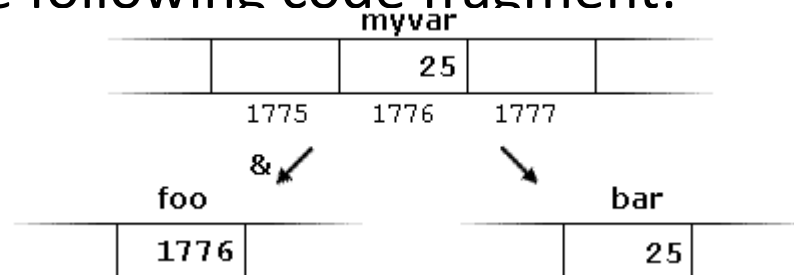
Overview

- In earlier chapters, variables have been explained as locations in the computer's memory which can be accessed by their identifier (their name). This way, the program does not need to care about the physical address of the data in memory; it simply uses the identifier whenever it needs to refer to the variable.
- For a C++ program, the memory of a computer is like a succession of memory cells, each one byte in size, and each with a unique address. These single-byte memory cells are ordered in a way that allows data representations larger than one byte to occupy memory cells that have consecutive addresses.
- When a variable is declared, the memory needed to store its value is assigned a specific location in memory (its memory address). Generally, C++ programs do not actively decide the exact memory addresses where its variables are stored.
- an operating system that decides the particular memory locations on runtime. However, it may be useful for a program to be able to obtain the address of a variable during runtime in order to access data cells that are at a certain position relative to it.

Reference/ Address-of operator (&)

- The address of a variable can be obtained by preceding the name of a variable with an ampersand sign (&), known as address-of operator. For example:
 `foo = &myvar;`
- This would assign the address of variable `myvar` to `foo`; by preceding the name of the variable `myvar` with the address-of operator (&), we are no longer assigning the content of the variable itself to `foo`, but its address.
- The actual address of a variable in memory cannot be known before runtime, but let's assume, in order to help clarify some concepts, that `myvar` is placed during runtime in the memory address 1776.
- In this case, consider the following code fragment:

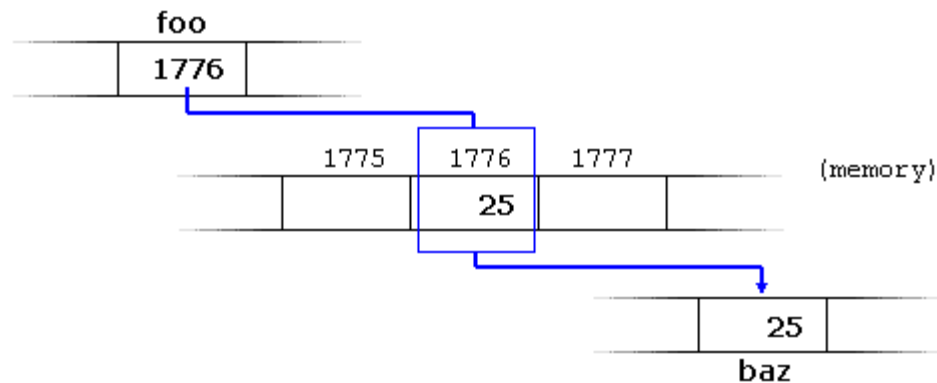
```
myvar = 25;  
foo = &myvar;  
bar = myvar;
```



Dereference/ value of operator (*)

- a variable which stores the address of another variable is called a pointer. Pointers are said to "point to" the variable whose address they store.
- An interesting property of pointers is that they can be used to access the variable they point to directly. This is done by preceding the pointer name with the dereference operator (*). The operator itself can be read as "value pointed to by". Example:

- `baz = *foo;`



Declaring pointers

- The declaration of pointers follows this syntax: `type *name;`
- where `type` is the data type pointed to by the pointer. This type is not the type of the pointer itself, but the type of the data the pointer points to. For example:
`int *number;`
`char *character;`
`double *decimals;`
- pointers occupy the same amount of space in memory (the size in memory of a pointer depends on the platform where the program runs).
- Note that the asterisk (*) used when declaring a pointer only means that it is a pointer (it is part of its type compound specifier), and should not be confused with the dereference operator.
- They are simply two different things represented with the same sign.

Example

```
int main ()
{
    int firstvalue, secondvalue;
    int * mypointer;
    mypointer = &firstvalue;
    *mypointer = 10;
    mypointer = &secondvalue;
    *mypointer = 20;
    cout << "firstvalue is " << firstvalue << '\n';
    cout << "secondvalue is " << secondvalue << '\n';
    return 0;}
```

Pointers and arrays

- The concept of arrays is related to that of pointers. In fact, arrays work very much like pointers to their first elements, and, actually, an array can always be implicitly converted to the pointer of the proper type. For example, consider these two declarations:

```
int myarray [20];
```

```
int * mypointer;
```

- The following assignment operation would be valid: `mypointer = myarray;`
- After that, `mypointer` and `myarray` would be equivalent and would have very similar properties. The main difference being that `mypointer` can be assigned a different address, whereas `myarray` can never be assigned anything, and will always represent the same block of 20 elements of type `int`. Therefore, the following assignment would not be valid:

```
myarray = mypointer;
```

example

```
// more pointers
#include <iostream>
using namespace std;
int main ()
{
    int numbers[5];
    int * p;
    p = numbers; *p = 10;
    p++; *p = 20;
```

```
p = &numbers[2]; *p = 30;
p = numbers + 3; *p = 40;
p = numbers; *(p+4) = 50;
for (int n=0; n<5; n++)
    cout << numbers[n] << ", ";
return 0;
}
```


Cont..

- Pointers and arrays support the same set of operations, with the same meaning for both. The main difference being that pointers can be assigned new addresses, while arrays cannot.
- In the chapter about arrays, brackets ([]) were explained as specifying the index of an element of the array. Well, in fact these brackets are a dereferencing operator known as offset operator. They dereference the variable they follow just as * does, but they also add the number between brackets to the address being dereferenced. For example:

`a[5] = 0;     // a [offset of 5] = 0`

`*(a+5) = 0;   // pointed to by (a+5) = 0`

- These two expressions are equivalent and valid, not only if a is a pointer, but also if a is an array. Remember that if an array, its name can be used just like a pointer to its first element.

Pointer initialization

- Pointers can be initialized to point to specific locations at the very moment they are defined:

```
int myvar;  
int * myptr = &myvar;
```

- The resulting state of variables after this code is the same as after:

```
int myvar;  
int * myptr;  
myptr = &myvar;
```

- Pointers can be initialized either to the address of a variable (such as in the case above), or to the value of another pointer (or array):

```
int myvar;  
int *foo = &myvar;  
int *bar = foo;
```

Pointer arithmetics

- To conduct arithmetical operations on pointers is a little different than to conduct them on regular integer types. To begin with, only addition and subtraction operations are allowed; the others make no sense in the world of pointers. But both addition and subtraction have a slightly different behavior with pointers, according to the size of the data type to which they point.
- Suppose the following ptrs point to the memory locations 1000, 2000, and 3000, respectively

`char *mychar;`

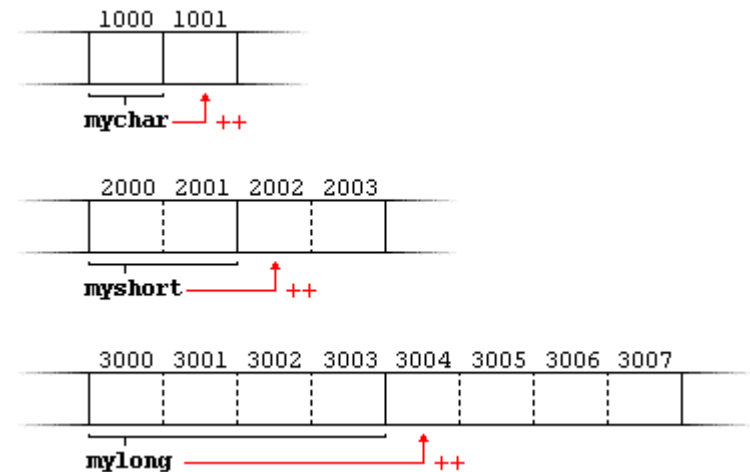
`short *myshort;`

`long *mylong;` then,

`++mychar;`

`++myshort;`

`++mylong;`



Pointers and const

- Pointers can be used to access a variable by its address, and this access may include modifying the value pointed. But it is also possible to declare pointers that can access the pointed value to read it, but not to modify it. For this, it is enough with qualifying the type pointed to by the pointer as const. For example:

```
int x;  
int y = 10;  
const int * p = &y;  
x = *p;      // ok: reading p  
*p = x;      // error: modifying p, which is const-qualified
```

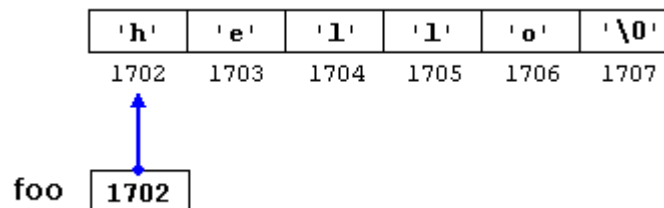
- pointer to non-const can be implicitly converted to a pointer to const. But not the other way around! As a safety feature, pointers to const are not implicitly convertible to pointers to non-const.

Pointers and string literals

- string literals are arrays containing null-terminated character sequences. In earlier sections, string literals have been used to be directly inserted into cout, to initialize strings and to initialize arrays of characters.
- But they can also be accessed directly. String literals are arrays of the proper array type to contain all its characters plus the terminating null-character, with each of the elements being of type const char (as literals, they can never be modified). For example:

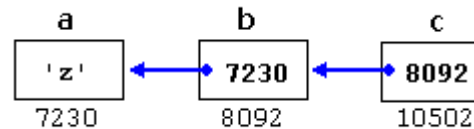
```
const char * foo = "hello";
```

- This declares an array with the literal representation for "hello", and then a pointer to its first element is assigned to foo. If we imagine that "hello" is stored at the memory locations that start at address 1702, we can represent the previous declaration as:



Pointers to pointers

- C++ allows the use of pointers that point to pointers, that these, in its turn, point to data (or even to other pointers). The syntax simply requires an asterisk (*) for each level of indirection in the declaration of the pointer:
- `char a;` `a = 'z';`
- `char * b;` `b = &a;`
- `char ** c;` `c = &b;`
- This, assuming the randomly chosen memory locations for each variable of 7230, 8092, and 10502, could be represented as:
- The new thing in this example is variable c, which is a pointer to a pointer, and can be used in three different levels of indirection, each one of them would correspond to a different value:
- c is of type `char**` and a value of 8092
- *c is of type `char*` and a value of 7230
- **c is of type `char` and a value of 'z'



void pointers

- The void type of pointer is a special type of pointer. In C++, void represents the absence of type. Therefore, void pointers are pointers that point to a value that has no type (and thus also an undetermined length and undetermined dereferencing properties).
- This gives void pointers a great flexibility, by being able to point to any data type, from an integer value or a float to a string of characters. In exchange, they have a great limitation: the data pointed to by them cannot be directly dereferenced (which is logical, since we have no type to dereference to), and for that reason, any address in a void pointer needs to be transformed into some other pointer type that points to a concrete data type before being dereferenced.

Invalid pointers and null pointers

- In principle, pointers are meant to point to valid addresses, such as the address of a variable or the address of an element in an array. But pointers can actually point to any address, including addresses that do not refer to any valid element. Typical examples of this are uninitialized pointers and pointers to nonexistent elements of an array:

```
int * p;           // uninitialized pointer (local variable)
int myarray[10];
int * q = myarray+20; // element out of bounds
```

- Neither `p` nor `q` point to addresses known to contain a value, but none of the above statements causes an error. In C++, pointers are allowed to take any address value, no matter whether there actually is something at that address or not. What can cause an error is to dereference such a pointer (i.e., actually accessing the value they point to). Accessing such a pointer causes undefined behavior, ranging from an error during runtime to accessing some random value.

Invalid pointers and null pointers

- But, sometimes, a pointer really needs to explicitly point to nowhere, and not just an invalid address. For such cases, there exists a special value that any pointer type can take: the null pointer value. This value can be expressed in C++ in two ways: either with an integer value of zero, or with the nullptr keyword:

```
int * p = 0;
```

```
int * q = nullptr;
```

```
int * r = NULL;
```

- NULL is defined in several headers of the standard library, and is defined as an alias of some null pointer constant value (such as 0 or nullptr).
- Do not confuse null pointers with void pointers! A null pointer is a value that any pointer can take to represent that it is pointing to "nowhere", while a void pointer is a type of pointer that can point to somewhere without a specific type. One refers to the value stored in the pointer, and the other to the type of data it points to.

Pointers to functions

- The typical use of this is for passing a function as an argument to another function. Pointers to functions are declared with the same syntax as a regular function declaration, except that the name of the function is enclosed between parentheses () and an asterisk (*) is inserted before the name:

Example

```
// pointer to functions
#include <iostream>
using namespace std;
int addition (int a, int b)
{ return (a+b); }
int subtraction (int a, int b)
{ return (a-b); }
int operation (int x, int y, int (*functocall)(int,int))
{
```

```
int g;
g = (*functocall)(x,y);
return (g);
}
int main ()
{
int m,n;
int (*minus)(int,int) = subtraction;
m = operation (7, 5, addition);
n = operation (20, m, minus);
cout <<n;
return 0;
}
```

